

Escuela de Ingeniería y Ciencias

Maestría en Inteligencia Artificial Aplicada

PRESENTA:

José Antonio Fernández Pérez · A01795991

Deménard Gardy Armand · A01797139

Luis Daniel Castillo Alegría · A01224696

Raúl Adrián Delgado Rodríguez · A01246414

ACTIVIDAD:

**Proyecto Final: Navegación Autónoma con Conditional Imitation Learning
Equipo 19**

Curso: Navegación autónoma

Profesor: Dr. David Antonio Torres

30 de junio de 2026

Resumen ejecutivo

Se implementó una solución de Conditional Imitation Learning (CIL) con tres comandos de alto nivel: STRAIGHT=0, LEFT=1 y RIGHT=2. El flujo corrige el conjunto original sin encabezado, excluye 321 muestras de recuperación, impide fuga entre entrenamiento y validación mediante una división por fuente y aumenta el conjunto hasta 12,956 muestras. La CNN condicionada alcanzó un MAE de validación global de 0.0220 rad, inferior al objetivo de 0.05 rad.

En Webots R2025a se integraron Camera Recognition, LiDAR, radar, giroscopio y tres sensores laterales. El árbitro de seguridad prioriza el frenado ante peatones, la evasión del autobús estacionado, el control de distancia y, finalmente, la dirección predicha por CIL. Se verificaron dos recorridos limpios por ruta, incluyendo evasión, frenado ante peatón y seguimiento de tráfico.

Video final publicado en YouTube: https://youtu.be/YObQTAm8_VM

1. Planteamiento y objetivos

El objetivo es conducir el vehículo autónomo por tres rutas urbanas mediante imitación condicional, manteniendo velocidad de evaluación de 22 km/h y un límite absoluto de 30 km/h. La solución debe reaccionar ante vehículos próximos, peatones y un autobús estacionado, además de mantener una ejecución reproducible.

Codevilla et al. propusieron condicionar la política de conducción con un comando de navegación de alto nivel. Su planteamiento original contempla Follow lane, Left, Right y Straight. Para la evaluación de este proyecto se adapta explícitamente el espacio de comandos a las tres rutas solicitadas y se elimina la etiqueta recovery; la recuperación ante obstáculos es una máquina de estados de seguridad, no una clase aprendida.

- Ruta recta: torres residenciales del noreste → silos; tráfico y evasión del autobús.
- Ruta derecha: Subway norte → iglesia; detección y frenado ante peatón.
- Ruta izquierda: parque infantil → Subway norte.

2. Preparación e integridad del conjunto de datos

El CSV original carecía de encabezado y contenía cuatro comandos. El proceso de saneamiento conserva sólo 0, 1 y 2, mueve las 321 filas recovery a un archivo histórico no publicado y comprueba tanto referencias faltantes como imágenes huérfanas. El conjunto canónico contiene 6,129 imágenes PNG.

Comando	Etiqueta	Muestras originales	Muestras de entrenamiento o aumentadas	Validación
0	Straight	1202	3848	240
1	Left	1895	3941	379
2	Right	3032	3941	607
Total		6129	11730	1226

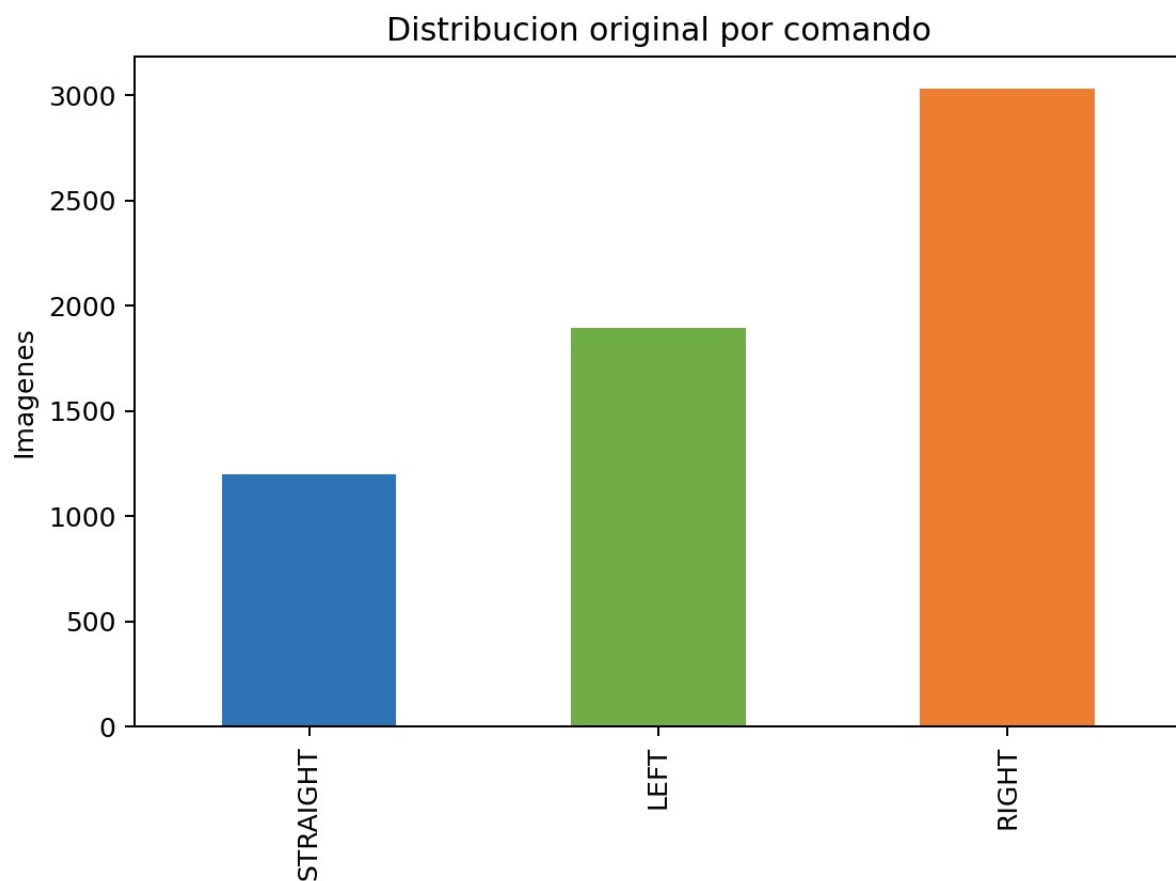


Figura 1. Distribución real de comandos antes y después del aumento.

2.1 División sin fuga y aumento

La división estratificada 80/20 se realiza sobre `source\_id` antes de cualquier transformación. Sólo después se generan reflejos horizontales —intercambiando LEFT

y RIGHT— y variaciones de brillo para reforzar STRAIGHT. La intersección de fuentes entre entrenamiento y validación es cero.

Comprobación	Resultado
Imágenes originales faltantes	0
Imágenes originales no referenciadas	0
Comandos válidos	0, 1, 2
Muestras después de aumento	12956
Fuentes compartidas train/val	0

3. Arquitectura y entrenamiento

La red recibe una imagen RGB preprocesada de $80 \times 160 \times 3$ y un vector one-hot de tres posiciones. Cinco capas convolucionales extraen características; la rama visual se aplana y concatena con el comando. Capas densas producen un único ángulo de dirección en radianes. Se emplearon semilla reproducible, Adam, pérdida MSE, MAE, EarlyStopping, ReduceLROnPlateau y checkpoint del mejor HDF5.

CNN condicionada por comando (modelo HDF5 real)

Capa	Tipo	Salida	Parametros total
image_input	InputLayer	(None, 80, 160, 3)	0
conv2d	Conv2D	(None, 38, 78, 24)	1,824
conv2d_1	Conv2D	(None, 17, 37, 36)	21,636
conv2d_2	Conv2D	(None, 7, 17, 48)	43,248
conv2d_3	Conv2D	(None, 5, 15, 64)	27,712
conv2d_4	Conv2D	(None, 3, 13, 64)	36,928
flatten	Flatten	(None, 2496)	0
command_input	InputLayer	(None, 3)	0
concatenate	Concatenate	(None, 2499)	0
dense	Dense	(None, 100)	250,000
dropout	Dropout	(None, 100)	0
dense_1	Dense	(None, 50)	5,050
dense_2	Dense	(None, 10)	510
steering_output	Dense	(None, 1)	11

Figura 2. Arquitectura obtenida directamente del modelo HDF5 entrenado.

Parámetro	Valor
Tamaño de entrada	80 × 160 × 3
Comando	One-hot de 3 clases
Optimizador	Adam
Pérdida	MSE
Callbacks	EarlyStopping, ReduceLROnPlateau, ModelCheckpoint
Formato de salida	HDF5 + model_metadata.json

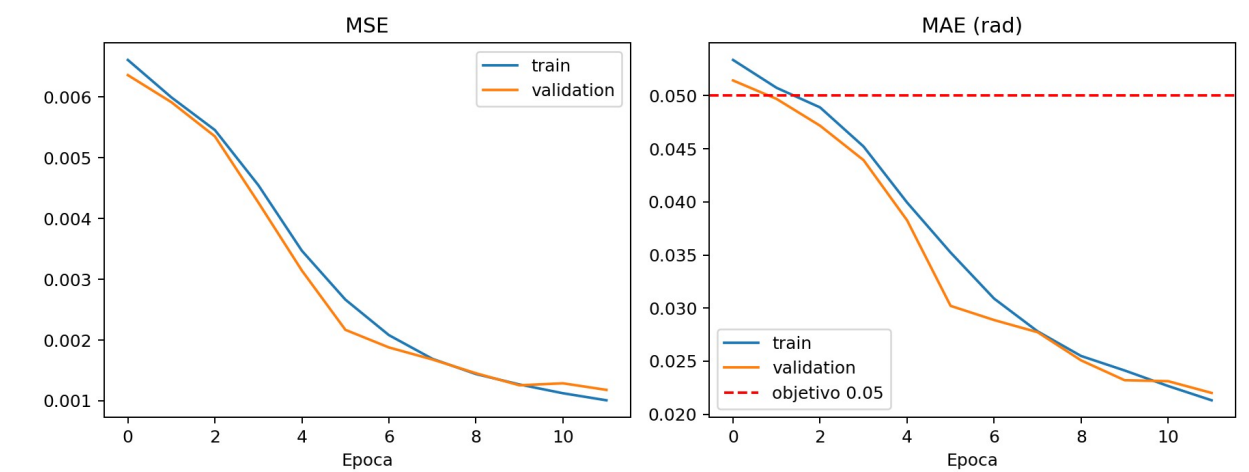


Figura 3. Curvas reales de pérdida y MAE durante el entrenamiento.

3.1 Resultados cuantitativos

Métrica	Resultado (rad)	Criterio
MAE global de validación	0.0220	Objetivo ≤ 0.05
MAE Straight	0.0152	Desglose por comando
MAE Left	0.0241	Desglose por comando
MAE Right	0.0234	Desglose por comando
MSE global	0.0012	Métrica complementaria

4. Controladores y portabilidad

4.1 Captura en World 1

El controlador de captura usa rutas relativas al proyecto, crea el encabezado ``image,steering,command``, limita la velocidad a 30 km/h y registra cada 100 ms. Las teclas W, A y D asignan los tres comandos válidos.

4.2 Control autónomo en World 2

El controlador carga ``cil_model.h5`` y ``model_metadata.json`` desde su propio directorio, replica el preprocesamiento del notebook, limita ángulo y tasa de cambio, y ejecuta inferencia cada cuatro pasos. El lanzador regenera presets relativos, crea el entorno Python, localiza SUMO y conecta el controlador externo.

Prioridad	Condición	Acción
1	Peatón ≤ 15 m por Recognition y LiDAR	Freno total
2	Autobús estacionado ≤ 26 m	Separación a 8 km/h, corredor $y=233.20$, límite $y=232.60$ y retorno a $y=236.40$
3	Vehículo entre 25 y 12 m	Regulación progresiva; alto ≤ 12 m; reanuda ≥ 15 m
4	Sin riesgo superior	Dirección CIL; 22 km/h

Antes del giro derecho, el controlador ejecuta un alto completo fuera de la intersección y permanece detenido hasta la fase verde del controlador CrossRoads; la corrida final liberó el vehículo en ``t=51.84 s``. El preset retrasa el ingreso del tráfico SUMO conflictivo hasta que el vehículo autónomo sale del cruce. En la ruta izquierda, el giro comienza dentro de la intersección y una asistencia lateral fija la salida en el carril derecho este, con centro ``y=39.60``.

4.3 Máquina de estados de evasión

Estado	Propósito	Criterio de transición
CIL	Conducción nominal	Autobús reconocido a ≤ 26 m

Estado	Propósito	Criterio de transición
EVASION_SEPARACION	Entrar al corredor legal $y=233.20$	Corredor, rumbo 0.30 rad o pared lateral
SEGUIMIENTO_PARED_DERECHA	Mantener separación sin cruzar $y=232.60$	Pared adquirida y luego ausente durante 12 pasos
RECUPERA_ORIENTACION	Retornar hacia el carril original	Rumbo <0.08 rad y $y \geq 235.60$, o 6 s
REINCORPORACION	Fijar el centro de carril $y=236.40$	Error lateral ≤ 0.45 m y rumbo <0.08 rad

5. Mundo, sensores y tráfico

World 2 incorpora Camera Recognition, Sick LMS 291, radar frontal, giroscopio y tres sensores de distancia en el costado derecho. `RecognizablePedestrian.proto` es local, tiene modelo `pedestrian`, colores de reconocimiento y objetos de colisión.

`SumoInterface` limita explícitamente la simulación a 30 vehículos.

Dispositivo	Uso
Camera + Recognition	Clasificar peatón/autobús y estimar distancia relativa
LiDAR	Confirmar obstáculo frontal en el campo de la cámara
Radar	Distancia y velocidad radial del vehículo precedente
Giroscopio	Recuperar orientación tras evasión
3 sensores laterales	Seguimiento de pared derecha y detección de fin de obstáculo

6. Presets de ruta y evidencia de Webots

Los presets se regeneran mediante `tools/create\_route\_worlds.py` y comparten la misma red vial. Cada uno fija pose inicial y comando, conservando rutas relativas. Una

cámara de evidencia independiente mantiene el vehículo visible con encuadre global estable y genera video H.264 Full HD (1920 × 1080 px, 30 fps).



Figura 4. Ruta recta: evidencia final de llegada al corredor de los silos.



Figura 5. Ruta derecha: cruce iniciado después del alto y de la fase verde.



Figura 6. Ruta izquierda: evidencia final en el corredor de Subway norte.

Ruta	Comando	Resultado técnico observado
Recto	0	Mínimo $y=232.77$ sin cruzar doble línea; retorno y permanencia en $y=236.40$; Llegada GPS= $(-188.06, 236.40)$
Derecha	2	Freno, espera hasta verde, cruce despejado y llegada GPS= $(28.95, -67.48)$
Izquierda	1	Salida al carril derecho este y llegada GPS= $(35.00, 39.60)$

Las tomas finales completaron las tres rutas por carril derecho, sin colisión ni U-turn. Los logs registran los cambios de estado y la llegada GPS; ffmpeg decodificó los tres clips sin errores. La revalidación corrigió la invasión del sentido opuesto durante la evasión, fijó la permanencia posterior en $y=236.40$, corrigió la entrada insegura al cruce y la salida del giro izquierdo al carril opuesto; conserva el filtro que distingue 'bus stop' de un autobús real.

7. Reproducción

Entrenamiento: abrir el notebook en Colab y ejecutar todas las celdas. La primera celda clona el repositorio público.

Comandos de aceptación local

```
git clone https://github.com/xak47d/proyecto-final-cil-equip019.git
cd proyecto-final-cil-equip019
./run_final_project.sh --setup
./run_final_project.sh --route straight --record
./run_final_project.sh --route right --record
./run_final_project.sh --route left --record
.venv-final/bin/python -m unittest discover -s tests -v
```

8. Uso de inteligencia artificial

Se utilizó OpenAI Codex basado en GPT-5 como asistente para generación de código, depuración, optimización, estructuración del notebook y del reporte, y preparación de pruebas. El equipo conserva responsabilidad sobre la revisión técnica, la ejecución en Webots, la interpretación de resultados y la entrega académica.

9. Conclusiones

La solución elimina correctamente la clase recovery y separa el aprendizaje de dirección de la recuperación operativa. El modelo supera la meta cuantitativa, el dataset queda trazable y libre de fuga, y el controlador integra sensores heterogéneos con prioridades verificables y completó dos recorridos limpios por ruta. El video final está publicado en https://youtu.be/YObQTAm8_VM.

Referencias

- Codevilla, F., Müller, M., López, A., Koltun, V. y Dosovitskiy, A. (2018). End-to-End Driving Via Conditional Imitation Learning. <https://arxiv.org/abs/1710.02410>
- CARLA Simulator. Imitation Learning — repositorio oficial. <https://github.com/carla-simulator/imitation-learning>
- Bojarski, M. et al. (2016). End to End Learning for Self-Driving Cars. arXiv:1604.07316.
- Cyberbotics. (2025). Webots R2025a Reference Manual. <https://cyberbotics.com/doc/reference/index>
- Eclipse SUMO. (2026). Simulation of Urban MObility documentation. <https://sumo.dlr.de/docs/>
- OpenAI. (2026). Codex (basado en GPT-5) [asistente de programación].

Anexo A. Controlador de captura completo

Código íntegro y comentado de `controllers/cil\_dataset\_recorder/cil\_dataset\_recorder.py`.

A.1 Fuente

```
"""Control manual y captura automatica del dataset CIL en World 1.

Comandos de navegacion:
    W -> STRAIGHT (0)
    A -> LEFT (1)
    D -> RIGHT (2)

Las flechas controlan el volante. La velocidad se conserva en 30 km/h, como
indica la actividad, y cada imagen se registra junto con el angulo de direccion
y el comando activo. Las rutas son relativas al proyecto para que el
controlador funcione en macOS, Windows o Linux sin editar el codigo.
"""

from __future__ import annotations

import csv
from pathlib import Path

from vehicle import Driver

STRAIGHT = 0
LEFT = 1
RIGHT = 2
COMMAND_NAMES = {STRAIGHT: "STRAIGHT", LEFT: "LEFT", RIGHT: "RIGHT"}

CRUISING_SPEED_KMH = 30.0
MAX_STEERING_RAD = 0.60
STEERING_STEP_RAD = 0.04
CAPTURE_PERIOD_MS = 100

def ensure_csv_header(csv_path: Path) -> None:
    """Create the log with the exact schema expected by the notebook."""
    if csv_path.exists() and csv_path.stat().st_size > 0:
        return
    csv_path.parent.mkdir(parents=True, exist_ok=True)
    with csv_path.open("w", newline="", encoding="utf-8") as handle:
        csv.writer(handle).writerow(["image", "steering_angle", "command"])

def main() -> None:
    driver = Driver()
    timestep = int(driver.getBasicTimeStep())

    camera = driver.getDevice("camera")
    camera.enable(timestep)
    keyboard = driver.getKeyboard()
    keyboard.enable(timestep)

    project_root = Path(__file__).resolve().parents[2]
    images_dir = project_root / "dataset" / "images"
    csv_path = project_root / "dataset" / "driving_log.csv"
```

```

images_dir.mkdir(parents=True, exist_ok=True)
ensure_csv_header(csv_path)

steering = 0.0
command = STRAIGHT
frame_index = 0
elapsed_since_capture = CAPTURE_PERIOD_MS

print("Captura CIL iniciada")
print("Flechas: volante | W: recto | A: izquierda | D: derecha")
print(f"Velocidad fija: {CRUISING_SPEED_KMH:.0f} km/h")

while driver.step() != -1:
    key = keyboard.getKey()
    while key != -1:
        if key == keyboard.LEFT:
            steering -= STEERING_STEP_RAD
        elif key == keyboard.RIGHT:
            steering += STEERING_STEP_RAD
        elif key in (ord("w"), ord("W")):
            command = STRAIGHT
            print("COMANDO: STRAIGHT")
        elif key in (ord("a"), ord("A")):
            command = LEFT
            print("COMANDO: LEFT")
        elif key in (ord("d"), ord("D")):
            command = RIGHT
            print("COMANDO: RIGHT")
        key = keyboard.getKey()

    steering = max(-MAX_STEERING_RAD, min(MAX_STEERING_RAD, steering))
    steering *= 0.98 # retorno gradual del volante cuando se suelta la tecla
    driver.setCruisingSpeed(CRUISING_SPEED_KMH)
    driver.setSteeringAngle(steering)

    elapsed_since_capture += timestep
    if elapsed_since_capture < CAPTURE_PERIOD_MS:
        continue
    elapsed_since_capture = 0

    sim_ms = int(driver.getTime() * 1000)
    image_name = f"img_{sim_ms:010d}_{frame_index:06d}.png"
    image_path = images_dir / image_name
    camera.saveImage(str(image_path), 100)

    with csv_path.open("a", newline="", encoding="utf-8") as handle:
        csv.writer(handle).writerow(
            [image_name, f"{steering:.8f}", command]
        )

    if frame_index % 25 == 0:
        print(
            f"Captura {frame_index}: {image_name} | "
            f"steering={steering:.3f} | {COMMAND_NAMES[command]}"
        )
    frame_index += 1

if __name__ == "__main__":
    main()

```

Anexo B. Controlador autónomo completo

Código íntegro y comentado de `controllers/cil\_autonomous\_driver/cil\_autonomous\_driver.py`.

B.1 Fuente

```
"""Controlador CIL y arbitro de seguridad para World 2.

Prioridad de control:
1. Peaton reconocido por la camara y confirmado por LiDAR: freno total.
2. Autobus estacionado reconocido y cercano: evasion por pared derecha.
3. Vehiculo detectado por radar: control de distancia longitudinal.
4. Direccion predicha por la CNN condicionada por el comando de navegacion.

El modulo mantiene las funciones matematicas independientes de Webots para que
los limites y umbrales puedan probarse automaticamente.
"""

from __future__ import annotations

import json
import math
import os
import time
from enum import Enum
from pathlib import Path
from typing import Iterable

import cv2
import numpy as np

STRAIGHT = 0
LEFT = 1
RIGHT = 2
COMMAND_NAMES = {STRAIGHT: "STRAIGHT", LEFT: "LEFT", RIGHT: "RIGHT"}
COMMAND_COUNT = 3

EVALUATION_SPEED_KMH = 22.0
MAX_SPEED_KMH = 30.0
MAX_STEERING_RAD = 0.60
PEDESTRIAN_STOP_DISTANCE_M = 15.0
PARKED_BUS_TRIGGER_DISTANCE_M = 26.0
FOLLOW_CONTROL_DISTANCE_M = 25.0
FOLLOW_STOP_DISTANCE_M = 12.0
FOLLOW_RESUME_DISTANCE_M = 15.0
AVOID_SPEED_KMH = 8.0
RECOVERY_SPEED_KMH = 10.0
WALL_PRESENT_DISTANCE_M = 3.7
WALL_TARGET_DISTANCE_M = 1.0
WALL_LOST_STEPS_REQUIRED = 12
BUS_PASS_TARGET_Y = 233.2
BUS_SEPARATOR_GUARD_Y = 232.6
BUS_NORMAL_LANE_Y = 236.4
BUS_SEPARATION_STEERING_RAD = -0.24
BUS_MAX_SEPARATION_HEADING_RAD = 0.30
MAX_STEERING_STEP_RAD = 0.035
INFERENCE_INTERVAL_STEPS = 4
ROUTE_TURN_SPEED_KMH = 10.0
SIGNAL_MIN_STOP_SECONDS = 2.0
```

```
RIGHT_SIGNAL_CYCLE_SECONDS = 41.856
```

```
class AvoidanceState(Enum):
    DRIVE = "CIL"
    SEPARATE_LEFT = "EVASION_SEPARACION"
    WALL_FOLLOW_RIGHT = "SEGUIMIENTO_PARED_DERECHA"
    RECOVER_HEADING = "RECUPERA_ORIENTACION"
    REJOIN = "REINCORPORACION"

def clamp(value: float, minimum: float, maximum: float) -> float:
    return max(minimum, min(maximum, value))

def command_to_one_hot(command: int) -> np.ndarray:
    """Encode one validated command for the model's second input."""
    if command not in COMMAND_NAMES:
        raise ValueError(f"Comando CIL invalido: {command}")
    encoded = np.zeros((1, COMMAND_COUNT), dtype=np.float32)
    encoded[0, command] = 1.0
    return encoded

def preprocess_bgra(image_bgra: np.ndarray, height: int = 80, width: int = 160) -> np.ndarray:
    """Apply exactly the crop/resize/normalization used during training."""
    if image_bgra.ndim != 3 or image_bgra.shape[2] != 4:
        raise ValueError(f"Imagen BGRA invalida: {image_bgra.shape}")
    rgb = cv2.cvtColor(image_bgra, cv2.COLOR_BGRA2RGB)
    crop_top = int(rgb.shape[0] * 0.25)
    cropped = rgb[crop_top:, :, :]
    resized = cv2.resize(cropped, (width, height), interpolation=cv2.INTER_AREA)
    normalized = resized.astype(np.float32) / 255.0
    return np.expand_dims(normalized, axis=0)

def limit_steering_step(target: float, current: float) -> float:
    delta = clamp(target - current, -MAX_STEERING_STEP_RAD, MAX_STEERING_STEP_RAD)
    return clamp(current + delta, -MAX_STEERING_RAD, MAX_STEERING_RAD)

def adaptive_follow_speed(distance_m: float) -> float:
    """Map radar distance to a safe target speed with a 12 m stop threshold."""
    if not math.isfinite(distance_m) or distance_m >= FOLLOW_CONTROL_DISTANCE_M:
        return EVALUATION_SPEED_KMH
    if distance_m <= FOLLOW_STOP_DISTANCE_M:
        return 0.0
    ratio = (distance_m - FOLLOW_STOP_DISTANCE_M) / (
        FOLLOW_CONTROL_DISTANCE_M - FOLLOW_STOP_DISTANCE_M
    )
    return clamp(EVALUATION_SPEED_KMH * ratio, 0.0, EVALUATION_SPEED_KMH)

def adaptive_follow_speed_hysteresis(
    distance_m: float, stopped_for_vehicle: bool
) -> tuple[float, bool]:
    """Mantiene el alto hasta que el vehiculo precedente vuelva a 15 m."""
    if distance_m <= FOLLOW_STOP_DISTANCE_M:
        return 0.0, True
    if stopped_for_vehicle and distance_m < FOLLOW_RESUME_DISTANCE_M:
        return 0.0, True
    return adaptive_follow_speed(distance_m), False
```

```

def route_turn_guidance(
    route: str,
    x: float,
    y: float,
    heading: float,
    turning: bool,
    completed: bool,
) -> tuple[float | None, bool, bool, int | None]:
    """Refuerza un único giro de ruta; fuera del cruce conserva la salida CIL."""
    if completed:
        return None, False, True, None
    if route == "left":
        # Enter deeper into the junction before steering so the exit arc lands
        # in the eastbound right lane instead of the opposing westbound lane.
        turning = turning or (y <= 49.0 and x < 5.0)
        if turning and heading >= 1.30:
            return None, False, True, STRAIGHT
        if turning:
            return -0.42, True, False, None
    elif route == "right":
        turning = turning or (y <= -58.0 and x > 20.0)
        if turning and heading <= -1.30:
            return None, False, True, STRAIGHT
        if turning:
            return 0.35, True, False, None
    return None, turning, completed, None

def route_destination_reached(route: str, x: float, y: float) -> bool:
    if route == "straight":
        return x <= -188.0 and y >= 220.0
    if route == "right":
        return x <= 29.0 and y <= -63.0
    if route == "left":
        return x >= 35.0 and 37.5 <= y <= 42.0
    return False

def route_heading_assist(route: str, heading: float, turn_completed: bool) -> float | None:
    """Mantiene el corredor de salida una vez completado el giro de ruta."""
    if route != "straight" and not turn_completed:
        return None
    target = {
        "straight": 0.0,
        "left": math.pi / 2.0,
        "right": -math.pi / 2.0,
    }.get(route)
    if target is None:
        return None
    return clamp(0.70 * (heading - target), -0.18, 0.18)

def route_corridor_assist(
    route: str,
    x: float,
    y: float,
    heading: float,
    turn_completed: bool,
) -> float | None:
    """Stabilize heading and place the car in the right-hand exit lane."""
    heading_steering = route_heading_assist(route, heading, turn_completed)
    if heading_steering is None:
        return None
    if route == "straight":

```

```

        return straight_lane_keeping_steering(y, heading)
    if route == "left":
        return clamp(heading_steering + 0.055 * (y - 39.6), -0.24, 0.24)
    return heading_steering

def route_signal_stop_required(
    route: str,
    x: float,
    y: float,
    turning: bool,
    completed: bool,
    released: bool,
) -> bool:
    """Hold the right-route vehicle at its stop line before entering the junction."""
    return (
        route == "right"
        and not turning
        and not completed
        and not released
        and 35.0 <= x <= 45.0
        and -40.0 < y <= -28.0
    )

def right_signal_is_green(simulation_time: float) -> bool:
    """Match the north/south straight-right phase of CrossRoadsTrafficLight."""
    phase = simulation_time % RIGHT_SIGNAL_CYCLE_SECONDS
    return 9.984 <= phase < 17.984

def route_approach_assist(
    route: str,
    x: float,
    y: float,
    heading: float,
    turning: bool,
    completed: bool,
) -> float | None:
    """Centra el auto en el carril antes del giro para entrar desde la geometria correcta."""
    if turning or completed:
        return None

    target_x = None
    if route == "right" and y > -58.0:
        target_x = 40.0
    elif route == "left" and y > 56.0:
        target_x = -50.4
    if target_x is None:
        return None
    lateral_error = x - target_x
    return clamp(0.055 * lateral_error + 0.90 * heading, -0.22, 0.22)

def front_lidar_distance(lidar) -> float:
    values = lidar.getRangeImage()
    if not values:
        return float("inf")
    center = len(values) // 2
    # Sector frontal de 90 grados, alineado con el campo de la camara.
    half_window = max(1, len(values) // 4)
    valid = [
        value
        for value in values[center - half_window : center + half_window]
        if math.isfinite(value) and 0.1 < value <= 50.0
    ]

```



```

]
return min(valid) if valid else float("inf")

def closest_radar_target(radar) -> tuple[float, float]:
    """Return distance and radial speed of the nearest target within +/-17 deg."""
    candidates = []
    for target in radar.getTargets():
        # The BmwX5 front slot can intermittently report a self-return near
        # 2.45 m; LiDAR still protects the immediate zone, so ignore <3 m.
        if abs(float(target.azimuth)) <= 0.30 and float(target.distance) >= 3.0:
            candidates.append((float(target.distance), float(target.speed)))
    return min(candidates, key=lambda item: item[0]) if candidates else (float("inf"), 0.0)

def closest_recognized(camera, model_fragment: str) -> float:
    distances = []
    fragment = model_fragment.lower()
    for obj in camera.getRecognitionObjects():
        model = str(obj.getModel()).lower()
        # Webots labels street furniture as "bus stop". A substring-only
        # comparison would therefore launch an avoidance manoeuvre against the
        # shelter itself. Only vehicle-like bus models are valid obstacles.
        if fragment not in model or (fragment == "bus" and "stop" in model):
            continue
        position = obj.getPosition()
        distance = math.sqrt(sum(float(axis) ** 2 for axis in position))
        image_x, image_y = obj.getPositionOnImage()
        if 0 <= image_x < camera.getWidth() and 0 <= image_y < camera.getHeight():
            distances.append(distance)
    return min(distances) if distances else float("inf")

def read_right_distances(sensors: dict[str, object]) -> dict[str, float]:
    result = {}
    for name, sensor in sensors.items():
        value = float(sensor.getValue())
        result[name] = value if math.isfinite(value) and value > 0.05 else float("inf")
    return result

def wall_present(distances: dict[str, float]) -> bool:
    return any(value < WALL_PRESENT_DISTANCE_M for value in distances.values())

def wall_following_steering(distances: dict[str, float]) -> float:
    valid = [value for value in distances.values() if math.isfinite(value)]
    if not valid:
        return -0.12
    front = distances["front"]
    middle = distances["middle"]
    rear = distances["rear"]
    side = middle if middle < WALL_PRESENT_DISTANCE_M else min(valid)
    distance_error = side - WALL_TARGET_DISTANCE_M
    alignment_error = (
        front - rear
        if front < WALL_PRESENT_DISTANCE_M and rear < WALL_PRESENT_DISTANCE_M
        else 0.0
    )
    return clamp(0.22 * distance_error + 0.16 * alignment_error, -0.42, 0.42)

def bus_separation_complete(
    y: float, heading: float, distances: dict[str, float]

```

```

) -> bool:
    """Stop moving left once the legal passing corridor or bus side is reached."""
    return (
        y <= BUS_PASS_TARGET_Y + 0.35
        or heading >= BUS_MAX_SEPARATION_HEADING_RAD
        or wall_present(distances)
    )

def update_wall_tracking(
    present: bool, seen: bool, lost_steps: int
) -> tuple[bool, int, bool]:
    """Count loss only after the right-side sensors have acquired the bus."""
    if present:
        return True, 0, False
    if not seen:
        return False, 0, False
    lost_steps += 1
    return True, lost_steps, lost_steps >= WALL_LOST_STEPS_REQUIRED

def straight_bus_corridor_steering(
    proposed_steering: float,
    y: float,
    heading: float,
    target_y: float = BUS_PASS_TARGET_Y,
) -> float:
    """Blend bus clearance with a hard guard against the center separator."""
    # GPS/heading establish the corridor; the side sensors only trim it. This
    # prevents an early long-range side return from pulling the car back toward
    # the bus before enough lateral clearance exists.
    corrected = 0.25 * proposed_steering + 0.55 * (target_y - y) + 1.10 * heading
    if y <= BUS_SEPARATOR_GUARD_Y + 0.20:
        corrected = max(corrected, 0.30)
    return clamp(corrected, -0.24, 0.34)

def straight_bus_rejoin_steering(y: float, heading: float) -> float:
    """Return smoothly from the passing corridor to the normal right lane."""
    lateral_error = BUS_NORMAL_LANE_Y - y
    target_heading = clamp(-0.10 * lateral_error, -0.22, 0.22)
    return clamp(heading - target_heading, -0.24, 0.24)

def straight_lane_keeping_steering(y: float, heading: float) -> float:
    """Continuously hold the straight route at its original lane center."""
    lateral_error = BUS_NORMAL_LANE_Y - y
    target_heading = clamp(-0.08 * lateral_error, -0.12, 0.12)
    return clamp(heading - target_heading, -0.18, 0.18)

def get_device(driver, name: str):
    try:
        return driver.getDevice(name)
    except BaseException as exc:
        raise RuntimeError(f"El mundo no contiene el dispositivo requerido: {name}") from exc

def main() -> None:
    import tensorflow as tf
    from vehicle import Driver

    driver = Driver()
    timestep = int(driver.getBasicTimeStep())

```

```

root = Path(__file__).resolve().parents[2]
model_path = Path(__file__).with_name("cil_model.h5")
metadata_path = Path(__file__).with_name("model_metadata.json")
if not model_path.exists() or not metadata_path.exists():
    raise FileNotFoundError("Faltan cil_model.h5 o model_metadata.json junto al controlador")

metadata = json.loads(metadata_path.read_text(encoding="utf-8"))
model = tf.keras.models.load_model(model_path, compile=False)

camera = get_device(driver, "camera")
camera.enable(timestep)
camera.recognitionEnable(timestep)
lidar = get_device(driver, "Sick LMS 291")
lidar.enable(timestep)
radar = get_device(driver, "front_radar")
radar.enable(timestep)
gyro = get_device(driver, "gyro")
gyro.enable(timestep)
gps = get_device(driver, "gps")
gps.enable(timestep)
keyboard = driver.getKeyboard()
keyboard.enable(timestep)

right_sensors = {
    "front": get_device(driver, "ds_right_front"),
    "middle": get_device(driver, "ds_right_middle"),
    "rear": get_device(driver, "ds_right_rear"),
}
for sensor in right_sensors.values():
    sensor.enable(timestep)

max_seconds = float(os.environ.get("CIL_MAX_SECONDS", "0") or 0)

command = int(os.environ.get("CIL_INITIAL_COMMAND", STRAIGHT))
route = os.environ.get("CIL_ROUTE", "").strip().lower()
if command not in COMMAND_NAMES:
    raise ValueError(f"CIL_INITIAL_COMMAND invalido: {command}")
state = AvoidanceState.DRIVE
state_started = driver.getTime()
heading = 0.0
previous_time = driver.getTime()
heading_before_avoidance = 0.0

wall_lost_steps = 0
wall_seen = False
steering_command = 0.0
cil_steering = 0.0
stopped_for_vehicle = False
route_turning = False
route_turn_completed = False
route_arrived = False
bus_avoidance_started = False
signal_stop_started = None
signal_released = False
frame = 0
wall_clock_started = time.monotonic()

print("Control CIL + seguridad iniciado")
print("W=STRAIGHT | A=LEFT | D=RIGHT")
print(
    "Umbrales: peaton=15 m, autobus=26 m, "
    "seguimiento=25 m, parada de seguimiento=12 m"
)

while driver.step() != -1:

```

```

now = driver.getTime()
dt = max(0.0, now - previous_time)
previous_time = now
heading += float(gyro.getValues()[2]) * dt

key = keyboard.getKey()
while key != -1:
    if key in (ord("W"), ord("w")):
        command = STRAIGHT
    elif key in (ord("A"), ord("a")):
        command = LEFT
    elif key in (ord("D"), ord("d")):
        command = RIGHT
    key = keyboard.getKey()

raw = camera.getImage()
if raw is None:
    continue
image_bgra = np.frombuffer(raw, np.uint8).reshape(
    (camera.getHeight(), camera.getWidth(), 4)
)
if frame % INFERENCE_INTERVAL_STEPS == 0:
    image_input = preprocess_bgra(
        image_bgra,
        int(metadata["image_height"]),
        int(metadata["image_width"]),
    )
    predicted = model.predict(
        [image_input, command_to_one_hot(command)], verbose=0
    )
    cil_steering = clamp(
        float(predicted[0][0]), -MAX_STEERING_RAD, MAX_STEERING_RAD
    )

lidar_distance = front_lidar_distance(lidar)
pedestrian_distance = closest_recognized(camera, "pedestrian")
bus_distance = closest_recognized(camera, "bus")
radar_distance, radar_speed = closest_radar_target(radar)
side_distances = read_right_distances(right_sensors)
x, y, _ = (float(value) for value in gps.getValues())
signal_holding = False

pedestrian_emergency = (
    pedestrian_distance <= PEDESTRIAN_STOP_DISTANCE_M
    and lidar_distance <= PEDESTRIAN_STOP_DISTANCE_M
)
if pedestrian_emergency:
    target_speed = 0.0
    target_steering = steering_command
    mode = "FRENO_PEATON"
else:
    if (
        state is AvoidanceState.DRIVE
        and not bus_avoidance_started
        and bus_distance <= PARKED_BUS_TRIGGER_DISTANCE_M
        and (not math.isfinite(radar_distance) or abs(radar_speed) < 0.75)
    ):
        bus_avoidance_started = True
        heading_before_avoidance = heading
        wall_lost_steps = 0
        wall_seen = False
        state = AvoidanceState.SEPARATE_LEFT
        state_started = now
        print(f">>> EVASION: autobus a {lidar_distance:.2f} m")

```

```

if state is AvoidanceState.SEPARATE_LEFT:
    target_speed = AVOID_SPEED_KMH
    target_steering = BUS_SEPARATION_STEERING_RAD
    if route == "straight":
        target_steering = straight_bus_corridor_steering(
            target_steering, y, heading
        )
    if bus_separation_complete(y, heading, side_distances):
        state = AvoidanceState.WALL_FOLLOW_RIGHT
        state_started = now
elif state is AvoidanceState.WALL_FOLLOW_RIGHT:
    target_speed = AVOID_SPEED_KMH
    target_steering = wall_following_steering(side_distances)
    if route == "straight":
        target_steering = straight_bus_corridor_steering(
            target_steering, y, heading
        )
    wall_seen, wall_lost_steps, wall_cleared = update_wall_tracking(
        wall_present(side_distances), wall_seen, wall_lost_steps
    )
    if wall_cleared:
        state = AvoidanceState.RECOVER_HEADING
        state_started = now
elif state is AvoidanceState.RECOVER_HEADING:
    heading_error = heading_before_avoidance - heading
    target_speed = RECOVERY_SPEED_KMH
    target_steering = clamp(-1.8 * heading_error, -0.38, 0.38)
    if route == "straight":
        target_steering = straight_bus_rejoin_steering(y, heading)
    if (
        abs(heading_error) < 0.08
        and (route != "straight" or y >= BUS_NORMAL_LANE_Y - 0.8)
    ) or now - state_started >= 6.0:
        state = AvoidanceState.REJOIN
        state_started = now
elif state is AvoidanceState.REJOIN:
    target_speed = RECOVERY_SPEED_KMH
    if route == "straight":
        target_steering = straight_bus_rejoin_steering(y, heading)
    else:
        target_steering = cil_steering
    if route == "straight":
        rejoin_complete = (
            abs(y - BUS_NORMAL_LANE_Y) <= 0.45 and abs(heading) < 0.08
        )
    else:
        rejoin_complete = now - state_started >= 2.0
    if rejoin_complete:
        state = AvoidanceState.DRIVE
        state_started = now
else:
    target_speed, stopped_for_vehicle = adaptive_follow_speed_hysteresis(
        radar_distance, stopped_for_vehicle
    )
    target_steering = cil_steering
    if route_signal_stop_required(
        route,
        x,
        y,
        route_turning,
        route_turn_completed,
        signal_released,
    ):

```

```

signal_holding = True
target_speed = 0.0
target_steering = route_approach_assist(
    route, x, y, heading, route_turning, route_turn_completed
) or 0.0
if abs(driver.getCurrentSpeed()) <= 0.5:
    if signal_stop_started is None:
        signal_stop_started = now
        print(">>> ALTO_SEMAFORO: vehiculo detenido en la linea")
    elif (
        now - signal_stop_started >= SIGNAL_MIN_STOP_SECONDS
        and right_signal_is_green(now)
    ):
        signal_released = True
        print(
            f">>> SEMAFORO_VERDE: inicia cruce seguro a t={now:.2f} s"
        )
    mode = "ALTO_SEMAFORO"
else:
    assist, route_turning, route_turn_completed, next_command = route_turn_guidance(
        route, x, y, heading, route_turning, route_turn_completed
    )
    if assist is not None:
        target_speed = min(target_speed, ROUTE_TURN_SPEED_KMH)
        target_steering = assist
        mode = "GIRO_RUTA"
    if next_command is not None:
        command = next_command
        print(f">>> GIRO COMPLETO: continua {COMMAND_NAMES[command]}")
    corridor_assist = route_corridor_assist(
        route, x, y, heading, route_turn_completed
    )
    approach_assist = route_approach_assist(
        route, x, y, heading, route_turning, route_turn_completed
    )
    if assist is None and approach_assist is not None:
        target_speed = min(target_speed, 16.0)
        target_steering = approach_assist
        mode = "APROXIMACION_RUTA"
    elif assist is None and corridor_assist is not None:
        if route == "straight" and (
            abs(y - 236.4) > 1.0 or abs(heading) > 0.10
        ):
            target_speed = min(target_speed, 10.0)
            target_steering = corridor_assist
            mode = "CORREDOR_RUTA"
    if route_destination_reached(route, x, y):
        target_speed = 0.0
        mode = "RUTA_COMPLETA"
        route_arrived = True
mode = state.value

if signal_holding and not signal_released and state is AvoidanceState.DRIVE:
    mode = "ALTO_SEMAFORO"
elif route_turning and state is AvoidanceState.DRIVE:
    mode = "GIRO_RUTA"
elif (route_turn_completed or route == "straight") and state is AvoidanceState.DRIVE:
    mode = "CORREDOR_RUTA"
if route_arrived:
    mode = "RUTA_COMPLETA"

target_speed = clamp(target_speed, 0.0, MAX_SPEED_KMH)
steering_command = limit_steering_step(target_steering, steering_command)
driver.setCruisingSpeed(target_speed)

```

```

        driver.setSteeringAngle(steering_command)

    if frame % 20 == 0:
        print(
            f"Modo={mode} | Cmd={COMMAND_NAMES[command]} | "
            f"steer={steering_command:.3f} | speed={target_speed:.1f} | "
            f"GPS=({x:.2f},{y:.2f}) | yaw={heading:.2f} | "
            f"LiDAR={lidar_distance:.2f} m | Radar={radar_distance:.2f} m | "
            f"({radar_speed:.2f} m/s) | peaton={pedestrian_distance:.2f} m | "
            f"bus={bus_distance:.2f} m"
        )
    frame += 1
    if max_seconds > 0 and now >= max_seconds:
        print(f"Fin automatico de evidencia a {now:.1f} s")
        break
    if route_arrived:
        print(f">>> RUTA COMPLETA {route}: GPS=({x:.2f},{y:.2f})")
        break
    if max_seconds > 0 and time.monotonic() - wall_clock_started >= max(60.0, max_seconds * 5.0):
        print("Fin de seguridad por limite de tiempo de reloj")
        break

if __name__ == "__main__":
    main()

```

Anexo C. Celdas de código del notebook

C.1 Celda 1

```

# Primera celda de codigo requerida por la actividad: clonar el dataset desde GitHub.
import os, sys
from pathlib import Path

IN_COLAB = "google.colab" in sys.modules
REPO_URL = "https://github.com/xak47d/proyecto-final-cil-equip019.git"
if IN_COLAB:
    PROJECT_ROOT = Path("/content/proyecto-final-cil-equip019")
    if not PROJECT_ROOT.exists():
        !git clone https://github.com/xak47d/proyecto-final-cil-equip019.git /content/proyecto-final-cil-equip019
        !pip install -q -r /content/proyecto-final-cil-equip019/requirements_colab.txt
else:
    PROJECT_ROOT = Path.cwd()

print("Proyecto:", PROJECT_ROOT)

```

C.2 Celda 2

```

import json, math, random, shutil
from collections import Counter
from pathlib import Path

import cv2
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import tensorflow as tf
from sklearn.model_selection import train_test_split
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau, ModelCheckpoint
from tensorflow.keras.layers import Input, Conv2D, Dense, Flatten, Dropout, Concatenate

```

```

from tensorflow.keras.models import Model
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.utils import Sequence, to_categorical

SEED = 19
random.seed(SEED)
np.random.seed(SEED)
tf.random.set_seed(SEED)

IMG_H, IMG_W = 80, 160
BATCH_SIZE = 32
EPOCHS = 12
COMMANDS = {0: "STRAIGHT", 1: "LEFT", 2: "RIGHT"}

RAW_DIR = PROJECT_ROOT / "dataset"
RAW_IMAGES = RAW_DIR / "images"
RAW_CSV = RAW_DIR / "driving_log.csv"
AUG_DIR = PROJECT_ROOT / "dataset_augmented"
AUG_IMAGES = AUG_DIR / "images"
AUG_CSV = AUG_DIR / "driving_log_augmented.csv"

FIGURES = PROJECT_ROOT / "docs" / "figures"
MODEL_DIR = PROJECT_ROOT / "controllers" / "cil_autonomous_driver"
FIGURES.mkdir(parents=True, exist_ok=True)
MODEL_DIR.mkdir(parents=True, exist_ok=True)

```

C.3 Celda 3

```

raw_df = pd.read_csv(RAW_CSV)
assert list(raw_df.columns) == ["image", "steering_angle", "command"]
assert set(raw_df["command"].unique()) == set(COMMANDS)
assert raw_df["image"].is_unique
missing = [name for name in raw_df["image"] if not (RAW_IMAGES / name).exists()]
assert not missing, f"Imagenes faltantes: {missing[:5]}"

raw_df["source_id"] = raw_df["image"].map(lambda name: Path(name).stem)
print("Registros originales:", len(raw_df))
print(raw_df["command"].map(COMMANDS).value_counts())
display(raw_df.head())

counts = raw_df["command"].map(COMMANDS).value_counts().reindex(COMMANDS.values())
ax = counts.plot(kind="bar", color=["#2E74B5", "#70AD47", "#ED7D31"])
ax.set_title("Distribucion original por comando")
ax.set_ylabel("Imagenes")
ax.set_xlabel("")
plt.tight_layout()
plt.savefig(FIGURES / "command_distribution_original.png", dpi=180)
plt.show()

```

C.4 Celda 4

```

train_raw, val_raw = train_test_split(
    raw_df,
    test_size=0.20,
    random_state=SEED,
    stratify=raw_df["command"],
)
train_sources = set(train_raw["source_id"])
val_sources = set(val_raw["source_id"])
assert train_sources.isdisjoint(val_sources)

expected_signature = {

```



```

"seed": SEED,
"raw_records": len(raw_df),
"train_sources": len(train_raw),
"validation_sources": len(val_raw),
"commands": COMMANDS,
}
signature_path = AUG_DIR / "augmentation_signature.json"
rebuild = True
if signature_path.exists() and AUG_CSV.exists():
    rebuild = json.loads(signature_path.read_text()) != expected_signature

if rebuild:
    if AUG_DIR.exists():
        shutil.rmtree(AUG_DIR)
    AUG_IMAGES.mkdir(parents=True, exist_ok=True)
    augmented_rows = []

    def save_variant(row, split, variant, image, steering, command):
        output_name = f"{split}_{variant}_{row.image}"
        ok = cv2.imwrite(str(AUG_IMAGES / output_name), image)
        if not ok:
            raise IOError(f"No se pudo escribir {output_name}")
        augmented_rows.append({
            "image": output_name,
            "steering_angle": float(steering),
            "command": int(command),
            "source_id": row.source_id,
            "split": split,
            "variant": variant,
        })

    for row in train_raw.itertuples(index=False):
        image = cv2.imread(str(RAW_IMAGES / row.image))
        save_variant(row, "train", "original", image, row.steering_angle, row.command)
        flipped_command = 2 if row.command == 1 else 1 if row.command == 2 else 0
        save_variant(row, "train", "flip", cv2.flip(image, 1), -row.steering_angle, flipped_command)
        if row.command == 0:
            darker = cv2.convertScaleAbs(image, alpha=0.75, beta=0)
            brighter_flip = cv2.convertScaleAbs(cv2.flip(image, 1), alpha=1.20, beta=0)
            save_variant(row, "train", "dark", darker, row.steering_angle, 0)
            save_variant(row, "train", "bright_flip", brighter_flip, -row.steering_angle, 0)

    for row in val_raw.itertuples(index=False):
        image = cv2.imread(str(RAW_IMAGES / row.image))
        save_variant(row, "validation", "original", image, row.steering_angle, row.command)

    augmented_df = pd.DataFrame(augmented_rows)
    AUG_DIR.mkdir(parents=True, exist_ok=True)
    augmented_df.to_csv(AUG_CSV, index=False)
    signature_path.write_text(json.dumps(expected_signature, indent=2))
else:
    augmented_df = pd.read_csv(AUG_CSV)

train_df = augmented_df.query("split == 'train'").reset_index(drop=True)
val_df = augmented_df.query("split == 'validation'").reset_index(drop=True)
assert set(train_df.source_id).isdisjoint(set(val_df.source_id))
assert len(augmented_df) > 10_000
print("Muestras finales:", len(augmented_df))
print("Entrenamiento:", len(train_df), "Validacion:", len(val_df))
print(train_df["command"].map(COMMANDS).value_counts())

```

C.5 Celda 5

```
def preprocess_image(path):
    image = cv2.imread(str(path))
    if image is None:
        raise FileNotFoundError(path)
    image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
    image = image[int(image.shape[0] * 0.25):, :, :]
    image = cv2.resize(image, (IMG_W, IMG_H), interpolation=cv2.INTER_AREA)
    return image.astype(np.float32) / 255.0

class CILSequence(Sequence):
    def __init__(self, dataframe, shuffle=True, **kwargs):
        super().__init__(**kwargs)
        self.df = dataframe.reset_index(drop=True)
        self.shuffle = shuffle
        self.indexes = np.arange(len(self.df))
        self.on_epoch_end()

    def __len__(self):
        return int(np.ceil(len(self.df) / BATCH_SIZE))

    def __getitem__(self, index):
        ids = self.indexes[index * BATCH_SIZE:(index + 1) * BATCH_SIZE]
        batch = self.df.iloc[ids]
        images = np.stack([preprocess_image(AUG_IMAGES / name) for name in batch.image])
        commands = to_categorical(batch.command.astype(int), num_classes=3)
        steering = batch.steering_angle.to_numpy(dtype=np.float32)
        return (images, commands), steering

    def on_epoch_end(self):
        if self.shuffle:
            np.random.shuffle(self.indexes)

train_gen = CILSequence(train_df, shuffle=True)
val_gen = CILSequence(val_df, shuffle=False)
sample_inputs, sample_targets = train_gen[0]
print(sample_inputs[0].shape, sample_inputs[1].shape, sample_targets.shape)
```

C.6 Celda 6

```
image_input = Input(shape=(IMG_H, IMG_W, 3), name="image_input")
x = Conv2D(24, 5, strides=2, activation="relu")(image_input)
x = Conv2D(36, 5, strides=2, activation="relu")(x)
x = Conv2D(48, 5, strides=2, activation="relu")(x)
x = Conv2D(64, 3, activation="relu")(x)
x = Conv2D(64, 3, activation="relu")(x)
x = Flatten()(x)

command_input = Input(shape=(3,), name="command_input")
x = Concatenate()([x, command_input])
x = Dense(100, activation="relu")(x)
x = Dropout(0.30)(x)
x = Dense(50, activation="relu")(x)
x = Dense(10, activation="relu")(x)
steering_output = Dense(1, name="steering_output")(x)

model = Model([image_input, command_input], steering_output, name="CIL_Equipo19")
model.compile(optimizer=Adam(1e-4), loss="mse", metrics=["mae"])
model.summary()
```

C.7 Celda 7

```
checkpoint = MODEL_DIR / "cil_model_best.h5"
callbacks = [
    EarlyStopping(monitor="val_mae", patience=3, restore_best_weights=True, mode="min"),
    ReduceLROnPlateau(monitor="val_loss", factor=0.5, patience=2, min_lr=1e-6),
    ModelCheckpoint(checkpoint, monitor="val_mae", save_best_only=True, mode="min"),
]
history = model.fit(
    train_gen,
    validation_data=val_gen,
    epochs=EPOCHS,
    callbacks=callbacks,
    verbose=1,
)
```

C.8 Celda 8

```
history_df = pd.DataFrame(history.history)
history_df.to_csv(FIGURES / "training_history.csv", index=False)
fig, axes = plt.subplots(1, 2, figsize=(10, 4))
axes[0].plot(history_df.loss, label="train")
axes[0].plot(history_df.val_loss, label="validation")
axes[0].set_title("MSE")
axes[0].set_xlabel("Epoca")
axes[0].legend()
axes[1].plot(history_df.mae, label="train")
axes[1].plot(history_df.val_mae, label="validation")
axes[1].axhline(0.05, color="red", linestyle="--", label="objetivo 0.05")
axes[1].set_title("MAE (rad)")
axes[1].set_xlabel("Epoca")
axes[1].legend()
plt.tight_layout()
plt.savefig(FIGURES / "training_curves.png", dpi=180)
plt.show()

predictions = model.predict(val_gen, verbose=0).reshape(-1)
actual = val_df.steering_angle.to_numpy(dtype=np.float32)
evaluation = []
for command, name in COMMANDS.items():
    mask = val_df.command.to_numpy() == command
    evaluation.append({
        "command": name,
        "samples": int(mask.sum()),
        "mae_rad": float(np.mean(np.abs(predictions[mask] - actual[mask]))),
    })
metrics = {
    "validation_mae_rad": float(np.mean(np.abs(predictions - actual))),
    "validation_mse": float(np.mean(np.square(predictions - actual))),
    "per_command": evaluation,
    "augmented_samples": int(len(augmented_df)),
    "train_samples": int(len(train_df)),
    "validation_samples": int(len(val_df)),
    "source_overlap": 0,
}
print(json.dumps(metrics, indent=2))
display(pd.DataFrame(evaluation))
```

C.9 Celda 9

```
final_model = MODEL_DIR / "cil_model.h5"
model.save(final_model)
metadata = {
    "image_height": IMG_H,
    "image_width": IMG_W,
    "channels": 3,
    "crop_top_ratio": 0.25,
    "normalization": "uint8/255.0",
    "commands": {str(key): value for key, value in COMMANDS.items()},
    "seed": SEED,
    "validation_mae_target_rad": 0.05,
}
(MODEL_DIR / "model_metadata.json").write_text(json.dumps(metadata, indent=2))
(MODEL_DIR / "training_metrics.json").write_text(json.dumps(metrics, indent=2))
print("Modelo:", final_model)
print("MAE validacion:", metrics["validation_mae_rad"])
assert metrics["validation_mae_rad"] <= 0.05, "El modelo no alcanza el objetivo MAE"
```

Anexo D. Guion del video

El guion dura aproximadamente 5:35 y distribuye la participación del equipo. Video final publicado en YouTube:

https://youtu.be/YObQTAm8_VM

D.1 Guion completo

```
# Guion del video demostrativo - Equipo 19
Video final publicado: https://youtu.be/YObQTAm8_VM
Duracion objetivo: **5:35**. El video final debe conservar la participacion
visible o audible de los tres integrantes y mostrar Optional Rendering para
camara, LiDAR, radar y sensores de distancia.
Integrantes y reparto:
- **Jose Antonio Fernandez Perez** - Apertura (0:00) y Ruta izquierda (3:40).
- **Demenard Gardy Armand** - Dataset y red (0:35) e Integracion (4:35).
- **Raul Adrian Delgado Rodriguez** - Ruta recta (1:35) y Ruta derecha (2:40).
Cada bloque incluye las senales visuales (**En pantalla**) y el texto a decir
(**Locucion**). El texto esta pensado para leerse a ritmo natural dentro del
tiempo indicado.
---
## 0:00-0:35 - Apertura - Jose Antonio Fernandez Perez
**En pantalla:** titulo del proyecto y nombres del equipo; corte rapido a World 1
y World 2 con el vehiculo en marcha.
**Locucion:**
> Hola, somos el Equipo 19. En este video mostramos un sistema de conduccion
> autonoma basado en aprendizaje por imitacion, o CIL, que responde a tres
> comandos de navegacion: seguir recto, girar a la izquierda y girar a la
> derecha. Trabajamos sobre dos escenarios de Webots: World 1, donde capturamos
> los datos, y World 2, donde validamos el modelo. La velocidad de captura fue de
> 30 kilometros por hora, y en World 2 evaluamos a 22 kilometros por hora para
> dar margen a los mecanismos de seguridad. A continuacion les mostramos como se
> construyo el conjunto de datos y la red.
---
## 0:35-1:35 - Dataset y red - Demenard Gardy Armand
**En pantalla:** mosaico de imagenes del dataset; diagrama de la CNN; curvas de
MSE y MAE de entrenamiento y validacion.
**Locucion:**
> Partimos de 6,129 imagenes originales y, tras el aumento de datos, llegamos a
> 12,956 muestras finales. El aumento consiste en un reflejo horizontal de la
> imagen: invertimos el signo del angulo de direccion e intercambiamos el comando
> izquierda por derecha. Es importante senalar que la division por identificador
> de origen se hace antes de aumentar, asi que no hay fuga de informacion entre
> entrenamiento y validacion. La red es una CNN con cinco capas convolucionales;
> el comando se inyecta como vector one-hot y se combina con capas densas que
> producen el angulo de direccion. El entrenamiento usa el optimizador Adam con
> tasa de aprendizaje de 1 por 10 a la menos 4, perdida MSE y metrica MAE, 12
> epocas, lotes de 32, y los callbacks EarlyStopping y ReduceLRonPlateau con
```

```
> factor 0.5 y paciencia 2. Fijamos la semilla en 19 y la entrada es de 160 por
> 80 por 3, con un recorte del 25 por ciento superior. El resultado: un MAE de
> validacion global de 0.022 radianes; por comando, 0.015 en recto, 0.024 en
> izquierda y 0.023 en derecha, todos por debajo del objetivo de 0.05 radianes.
---

## 1:35-2:40 - Ruta recta - Raul Adrian Delgado Rodriguez
**En pantalla:** consola con `Cmd=STRAIGHT`; vista de radar y LiDAR; overlay del
GPS y del limite `y=232.60`.
**Locucion:**
> Esta es la ruta recta. Salimos de las torres residenciales del noreste con
> destino a los silos, y en consola se mantiene el comando STRAIGHT. La gestion
> de obstaculos frontales usa el radar con tres umbrales: empieza a regular la
> velocidad desde 25 metros, se detiene por completo a 12 metros y reanuda la
> marcha a 15 metros. Para evasion lateral el umbral esta configurado en 26
> metros; en esta corrida el autobus se reconocio a 23.74 metros y la maniobra
> inicio con 16.62 metros medidos por LiDAR. Observen el seguimiento de la pared
> derecha: esta contenido por el limite y igual a 232.60, y el minimo que
> registramos fue 232.77, sin tocar ni cruzar la doble linea. Despues de
> rebasar, el vehiculo se reincorpora y permanece en el centro del carril
> original, en y igual a 236.40, hasta llegar al destino con GPS en menos 188.06,
> 236.40.
---

## 2:40-3:40 - Ruta derecha y peaton - Raul Adrian Delgado Rodriguez
**En pantalla:** consola con `Cmd=RIGHT`; giro a la derecha; camara detectando al
peaton; distancia frontal del LiDAR y velocidad.
**Locucion:**
> Seguimos con la ruta a la derecha. Salimos del Subway norte hacia la iglesia, y
> el comando RIGHT genera un giro limpio a la derecha, sin vuelta en U. En esta
> ruta aparece un peaton: la camara lo reconoce y el LiDAR mide la distancia
> frontal. Con el umbral de 15 metros, el vehiculo baja a 0 kilometros por hora
> antes del cruce. Hace el alto antes de la interseccion y espera la fase verde
> del semaforo. Solo continua cuando el peaton deja la trayectoria y el semaforo
> habilita el cruce; en ese momento la navegacion retoma el control.
---

## 3:40-4:35 - Ruta izquierda - Jose Antonio Fernandez Perez
**En pantalla:** consola con `Cmd=LEFT`; giro a la izquierda; permanencia en el
carril derecho con el volante controlado por la CNN.
**Locucion:**
> Esta es la ruta a la izquierda. Salimos del parque infantil hacia el Subway
> norte; el comando LEFT ejecuta el giro y el vehiculo se mantiene en el carril
> derecho. Quiero destacar un punto de diseno: el volante lo sigue controlando la
> CNN en todo momento, salvo cuando el arbitro de seguridad necesita una anulacion
> temporal. Es decir, el modelo aprendido conduce, y la seguridad solo interviene
> cuando hace falta.
---

## 4:35-5:15 - Integracion y prioridades - Demeard Gardy Armand
**En pantalla:** diagrama del arbitro de seguridad con sus entradas (peaton,
autobus, radar, CIL); panel de SUMO con el trafico.
**Locucion:**
> Veamos como se integra todo. Este es el arbitro de seguridad, que prioriza
> cuatro entradas: peaton, autobus, radar y el comando CIL. La idea central es
> que la seguridad solo sustituye temporalmente la velocidad o la direccion; el
> comando de navegacion nunca se pierde, siempre se conserva. Asi combinamos un
> control aprendido con reglas de seguridad explicitas. Ademas, el trafico de
> SUMO esta configurado con 30 vehiculos, que no es menos de lo solicitado para
> la evaluacion.
---

## 5:15-5:35 - Cierre - Jose Antonio Fernandez Perez
**En pantalla:** los tres integrantes en pantalla o en cuadro; enlace del
repositorio y conclusiones.
**Locucion:**
> En resumen, logramos una conduccion autonoma por imitacion que responde a tres
> comandos y respeta los limites del carril, con un MAE de validacion de 0.022
> radianes y un arbitro de seguridad que protege sin perder la navegacion.
> Gracias por su atencion; el codigo esta disponible en el repositorio que
> aparece en pantalla.
Cerrar antes de 6:00.
```